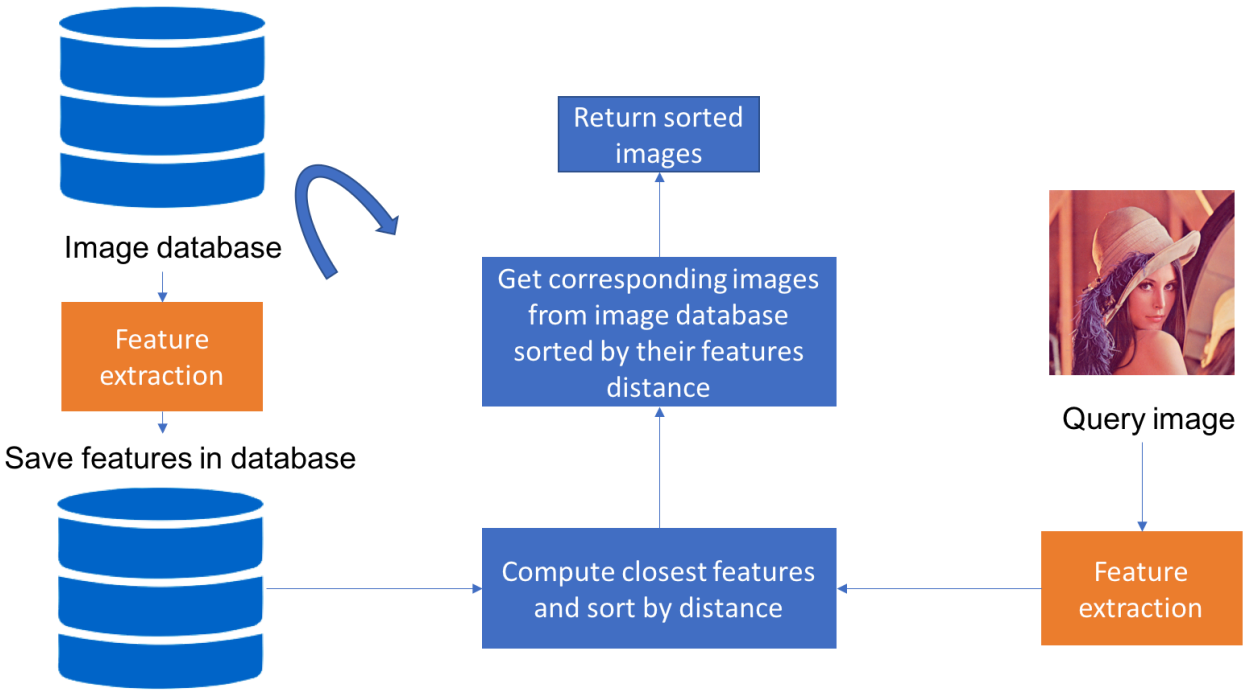
Objectif

La recherche d'images basée sur le contenu (CBIR) est une méthode permettant de rechercher des images à partir des caractéristiques visuelles, c'est-à-dire induites par les pixels. L'utilisation la plus connue de cette méthode est la fonction de recherche d'images du moteur de recherche Google. Elle s'effectue en quatre étapes :

1. Extraction des caractéristiques d'une base de données d'images pour former une base de données des caractéristiques,
2. Extraction des caractéristiques de l'image fournie en entrée,
3. Recherche des caractéristiques les plus similaires dans la base de données,
4. Retour de l'image associée aux caractéristiques trouvées.



Cette méthode répose sur l'algorithme d'extraction et la comparaison des caractéristiques.

Mesure de similarité

La recherche est exacte (force brute) : les écarts viennent des vecteurs et de la mesure, pas d'un index approximatif.

- Distance cosinus : 1 - similarité cosinus, ne compare que les directions
- Distance de manhattan

- Distance euclidienne

Algorithme d'extraction

- AKAZE et ORB, sous forme de sac de mots visuels : vocabulaire de 512 mots appris sur la base d'images, affectation des descripteurs binaires par distance de Hamming

Réseaux pré-entraînés chargés avec timm, chacun avec son pré-traitement prescrit :

- VGG16, dernière couche dense avant la classification (4096)
- NasNet Large, moyenne globale (4032)
- EfficientNet B7, moyenne globale (2560)
- ConvNeXt Tiny, CNN moderne de taille modeste (768)
- DINOv2 ViT-B/14, auto-supervisé (768)
- SigLIP 2 ViT-B/16, pré-entraîné sur des paires image-texte (768)

L'objectif de trouver la bonne combinaisons (algorithme d'extraction & mesure de similarité) qui permet d'avoir des réponses pertinentes.

Jeux de données

Nous avons utilisé la version 6 du jeu de données Apparel pour notre évaluation, découpé en une base d'images (`train` , 67 %) et des requêtes (`test` , 33 %), stratifiés par catégorie.

```
Elle contient 11385 images réparties en 24 catégories, divisées en deux jeux :
- base d'images : 7627 images
- requêtes : 3758 images
```

Evaluation

Couverture des extracteurs

Toutes les méthodes reçoivent les mêmes images. Une image sans vecteur (aucun point clé détecté) reste dans l'évaluation : absente de la base, elle ne peut pas être retournée ; requête, elle obtient un score nul.

method	base_sans_vecteur	requetes_sans_vecteur
AKAZE	4	4
ORB	0	0
VGG	0	0
NasNet	0	0
EfficientNet	0	0
ConvNeXt	0	0
DINOv2	0	0
SigLIP2	0	0

Nombre d'images sans vecteur par méthode, dans la base et parmi les requêtes.

Métriques d'évaluation

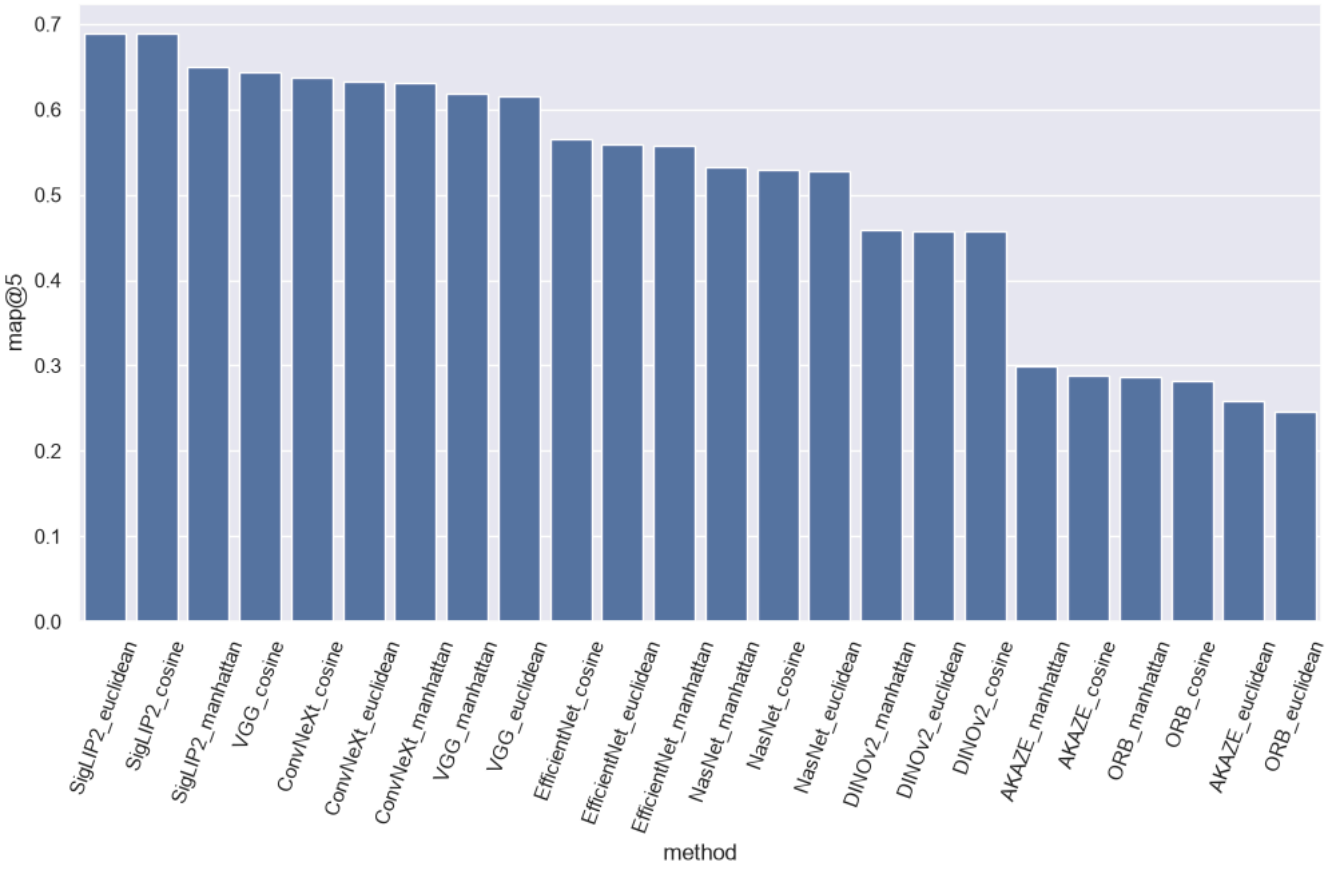
Chaque requête retourne ses 5 plus proches images.

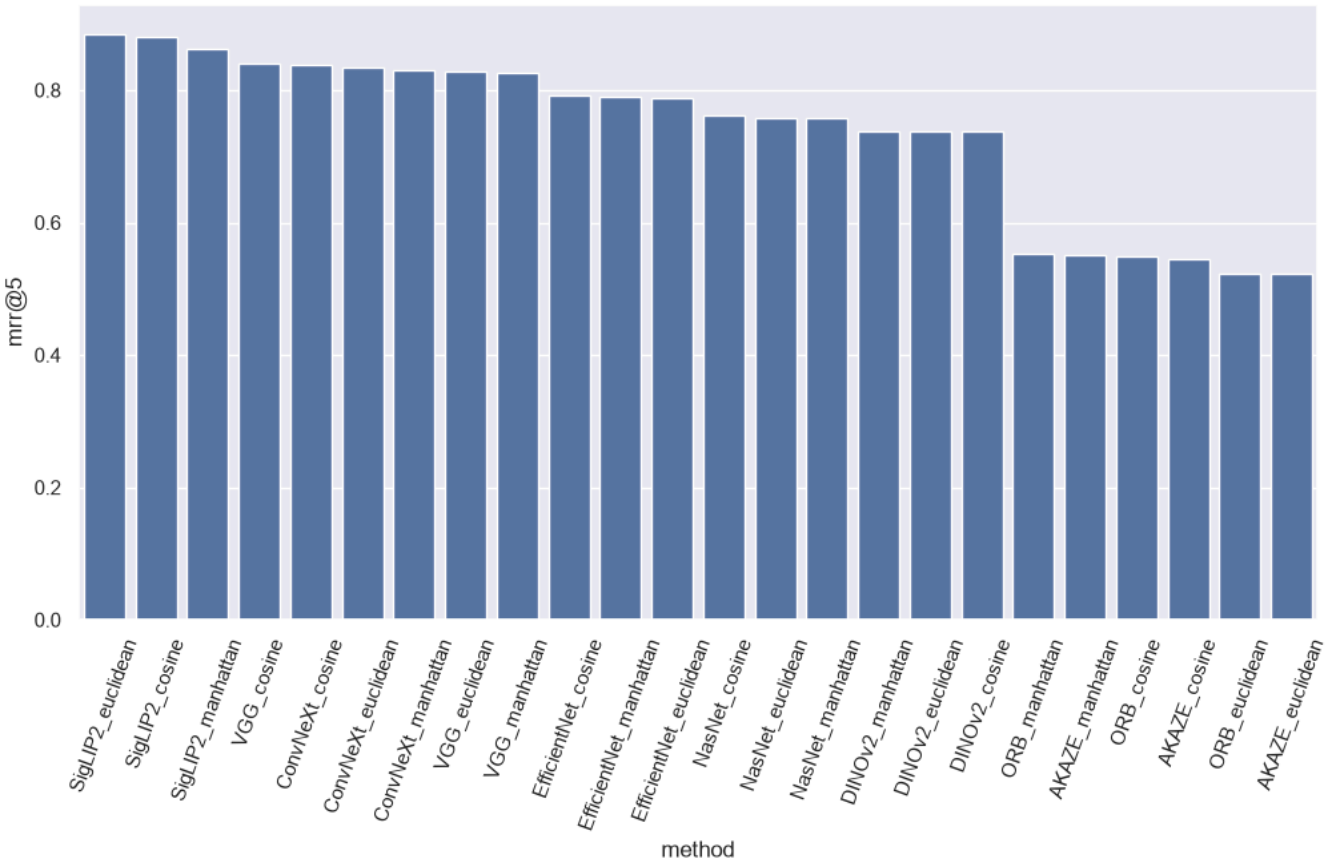
- MAP@5 : précision moyenne sur les 5 résultats, divisée par `min(images pertinentes dans la base, 5)`
- MRR@5 : inverse du rang du premier résultat pertinent, 0 au-delà du rang 5
- Précision au premier rang
- Temps d'une requête (ms) : temps moyen d'extraction d'une image seule + temps moyen de recherche

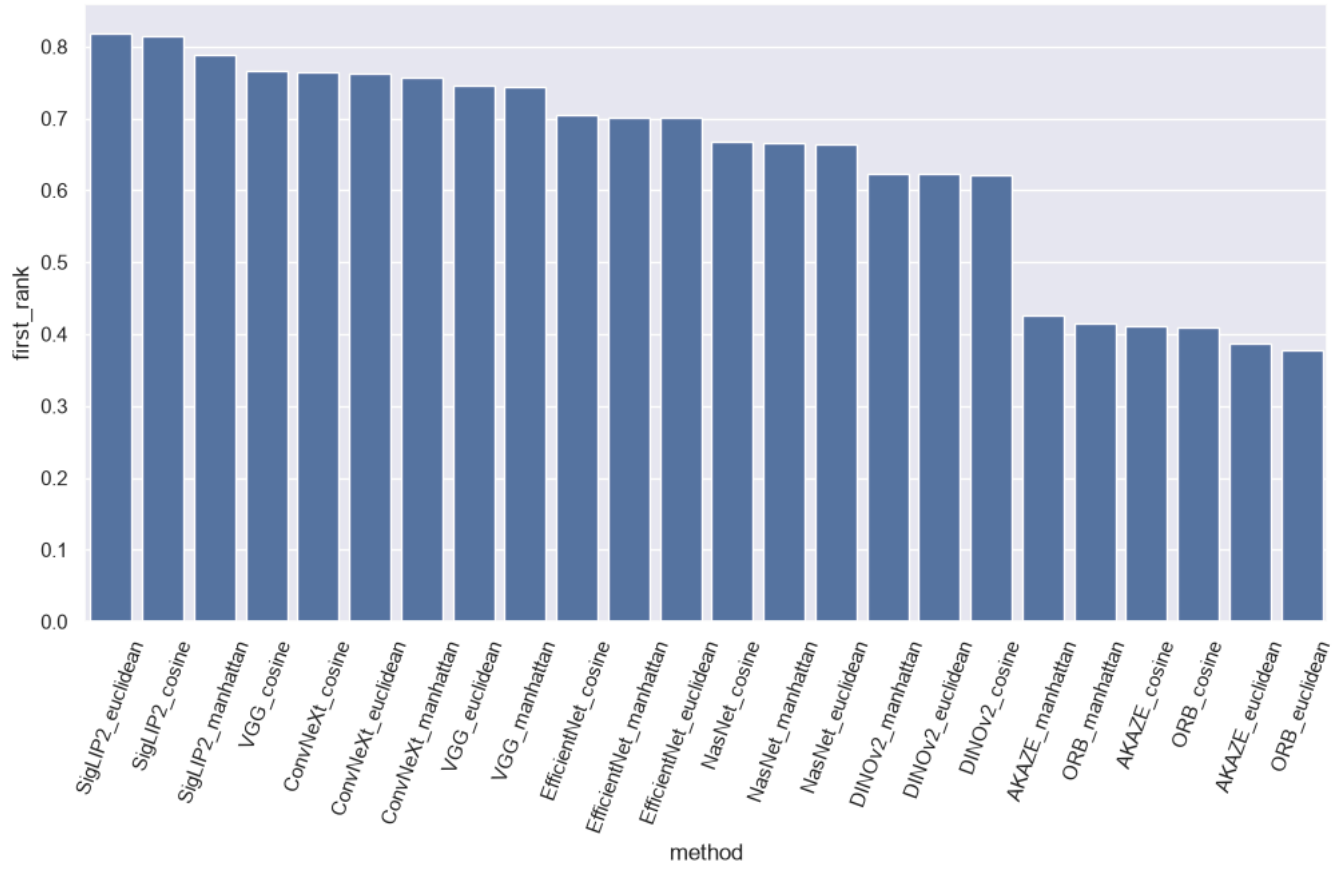
Pour plus de détail voir

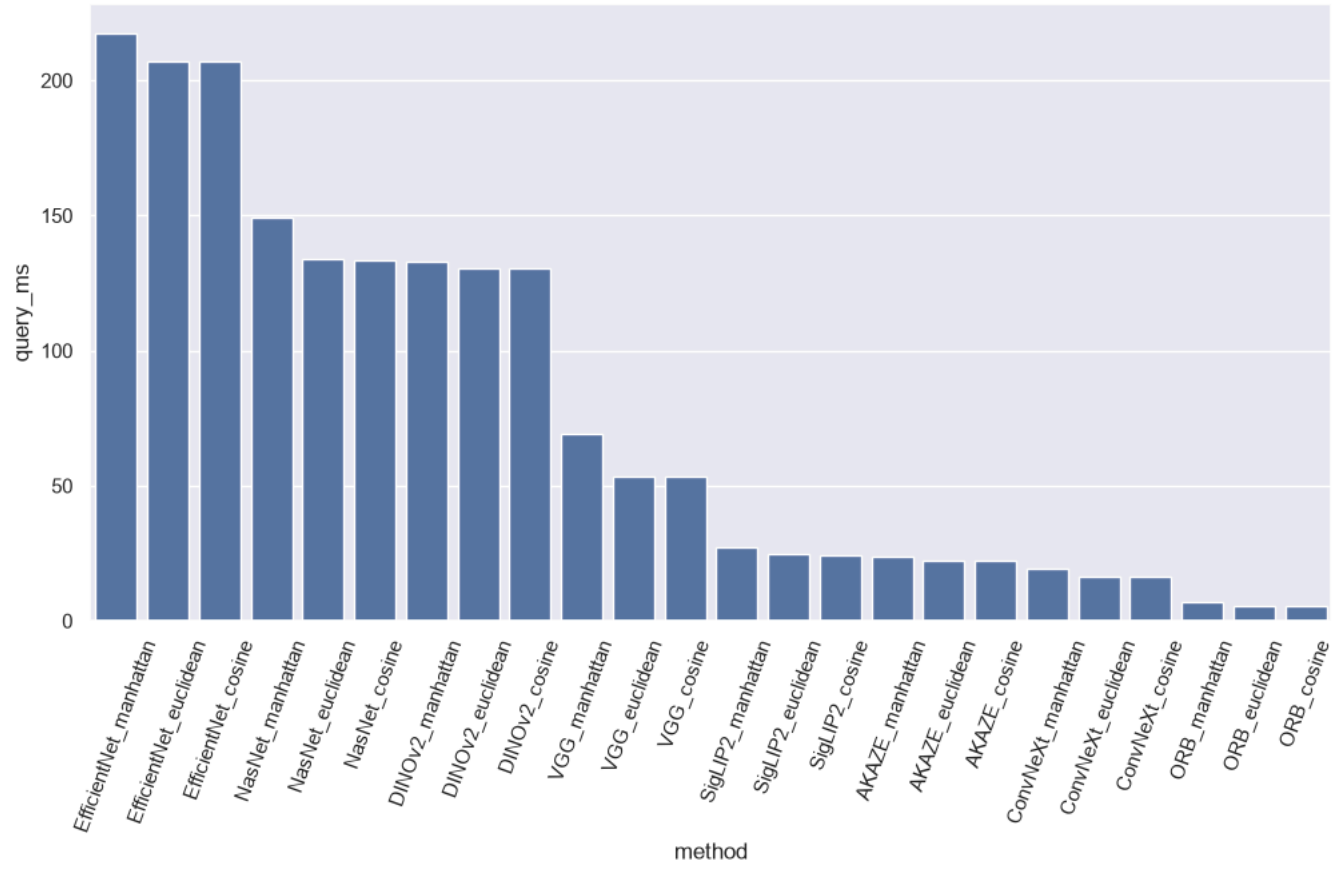
Colors

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.28856	0.543831	0.410857	22.137411
AKAZE_euclidean	0.258061	0.522317	0.38744	22.153636
AKAZE_manhattan	0.298627	0.551597	0.425226	23.839576
ORB_cosine	0.281823	0.548599	0.40926	5.47782
ORB_euclidean	0.245385	0.523479	0.377861	5.49867
ORB_manhattan	0.287037	0.552621	0.415647	7.196496
VGG_cosine	0.643959	0.840305	0.765833	53.282498
VGG_euclidean	0.614758	0.827328	0.745875	53.519908
VGG_manhattan	0.61835	0.826721	0.743747	68.997572
NasNet_cosine	0.529014	0.761806	0.667376	133.562377
NasNet_euclidean	0.527342	0.75746	0.663119	133.76933
NasNet_manhattan	0.531975	0.757189	0.666046	149.274009
EfficientNet_cosine	0.565959	0.791658	0.704364	206.975264
EfficientNet_euclidean	0.559463	0.788611	0.701171	207.112004
EfficientNet_manhattan	0.557699	0.788957	0.701969	217.32554
ConvNeXt_cosine	0.637493	0.838309	0.763438	16.464562
ConvNeXt_euclidean	0.631926	0.833897	0.762374	16.498031
ConvNeXt_manhattan	0.630635	0.830876	0.757318	19.152129
DINOv2_cosine	0.457729	0.736948	0.620809	130.376496
DINOv2_euclidean	0.457823	0.737099	0.622139	130.406067
DINOv2_manhattan	0.458408	0.73744	0.622938	133.059558
SigLIP2_cosine	0.688277	0.880814	0.814529	24.507493
SigLIP2_euclidean	0.688769	0.884034	0.817988	24.539723
SigLIP2_manhattan	0.649617	0.862387	0.788984	27.17785


map@5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
SigLIP2_euclidean
SigLIP2_cosine
SigLIP2_manhattan
VGG_cosine
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_manhattan
VGG_euclidean
EfficientNet_cosine
EfficientNet_euclidean
EfficientNet_manhattan
NasNet_manhattan
NasNet_cosine
NasNet_euclidean
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

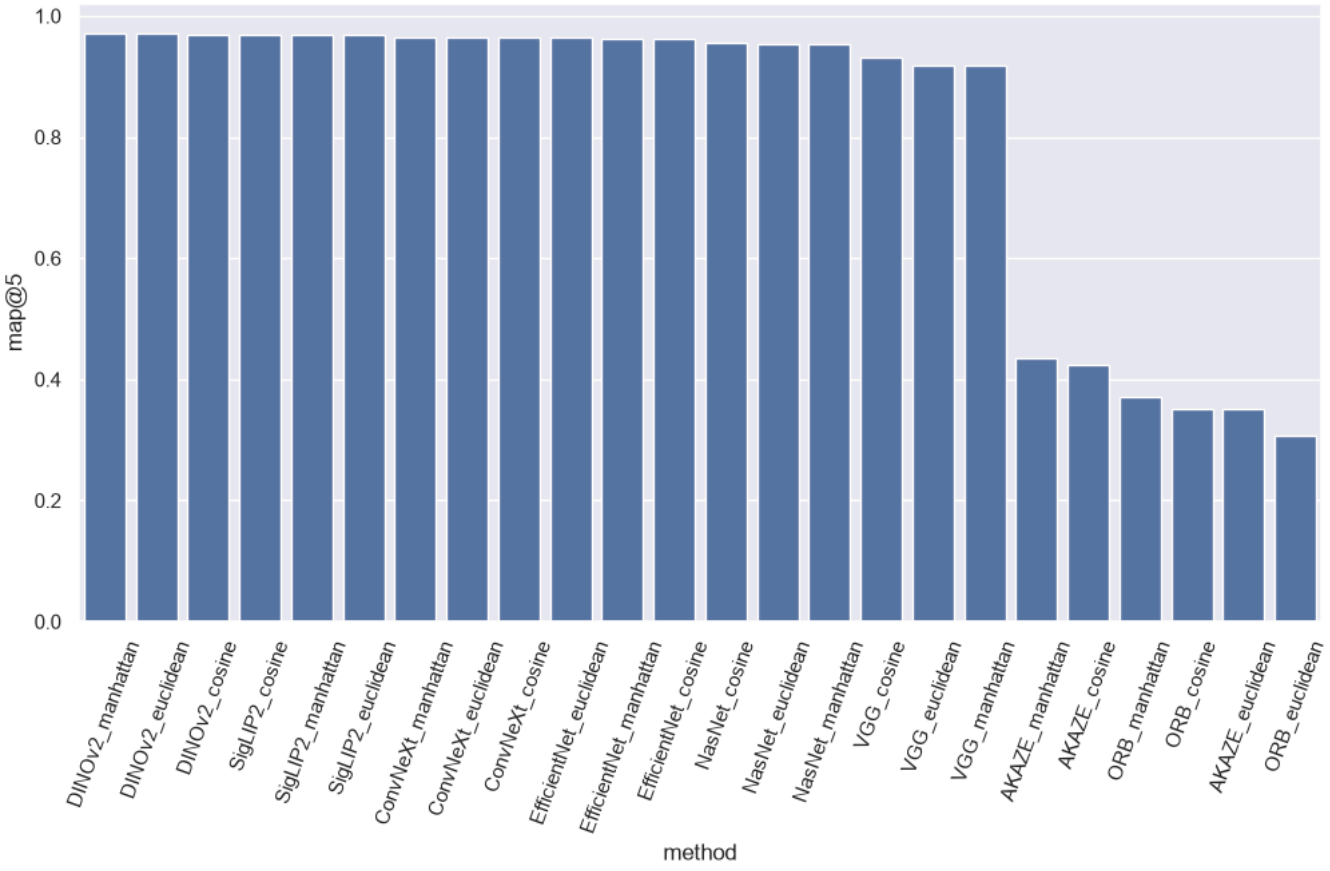

mrr@5
0.0
0.2
0.4
0.6
0.8
SigLIP2_euclidean
SigLIP2_cosine
SigLIP2_manhattan
VGG_cosine
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_euclidean
VGG_manhattan
EfficientNet_cosine
EfficientNet_manhattan
EfficientNet_euclidean
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
ORB_manhattan
AKAZE_manhattan
ORB_cosine
AKAZE_cosine
ORB_euclidean
AKAZE_euclidean
method

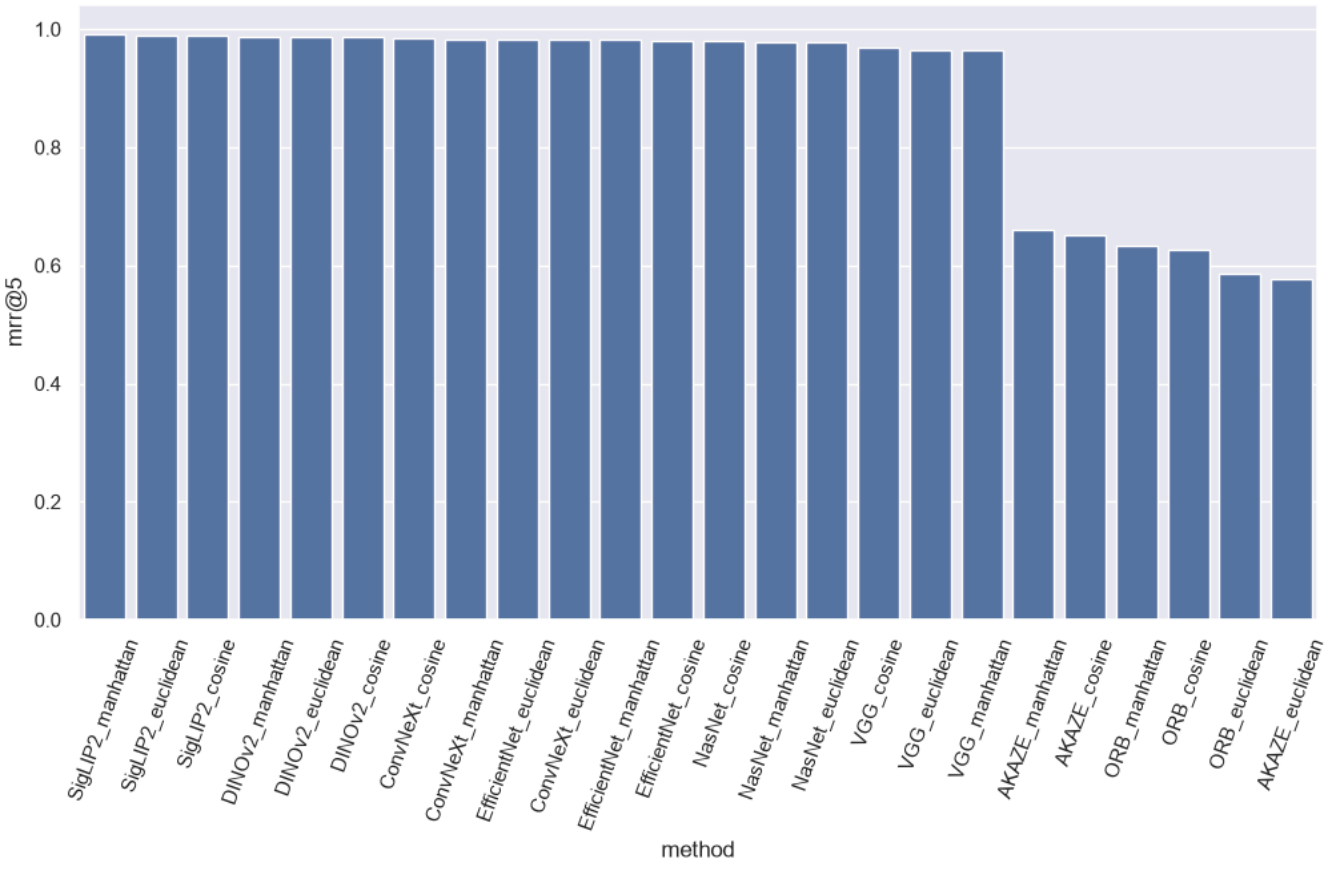

first_rank
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
SigLIP2_euclidean
SigLIP2_cosine
SigLIP2_manhattan
VGG_cosine
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_euclidean
VGG_manhattan
EfficientNet_cosine
EfficientNet_manhattan
EfficientNet_euclidean
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
AKAZE_manhattan
ORB_manhattan
AKAZE_cosine
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

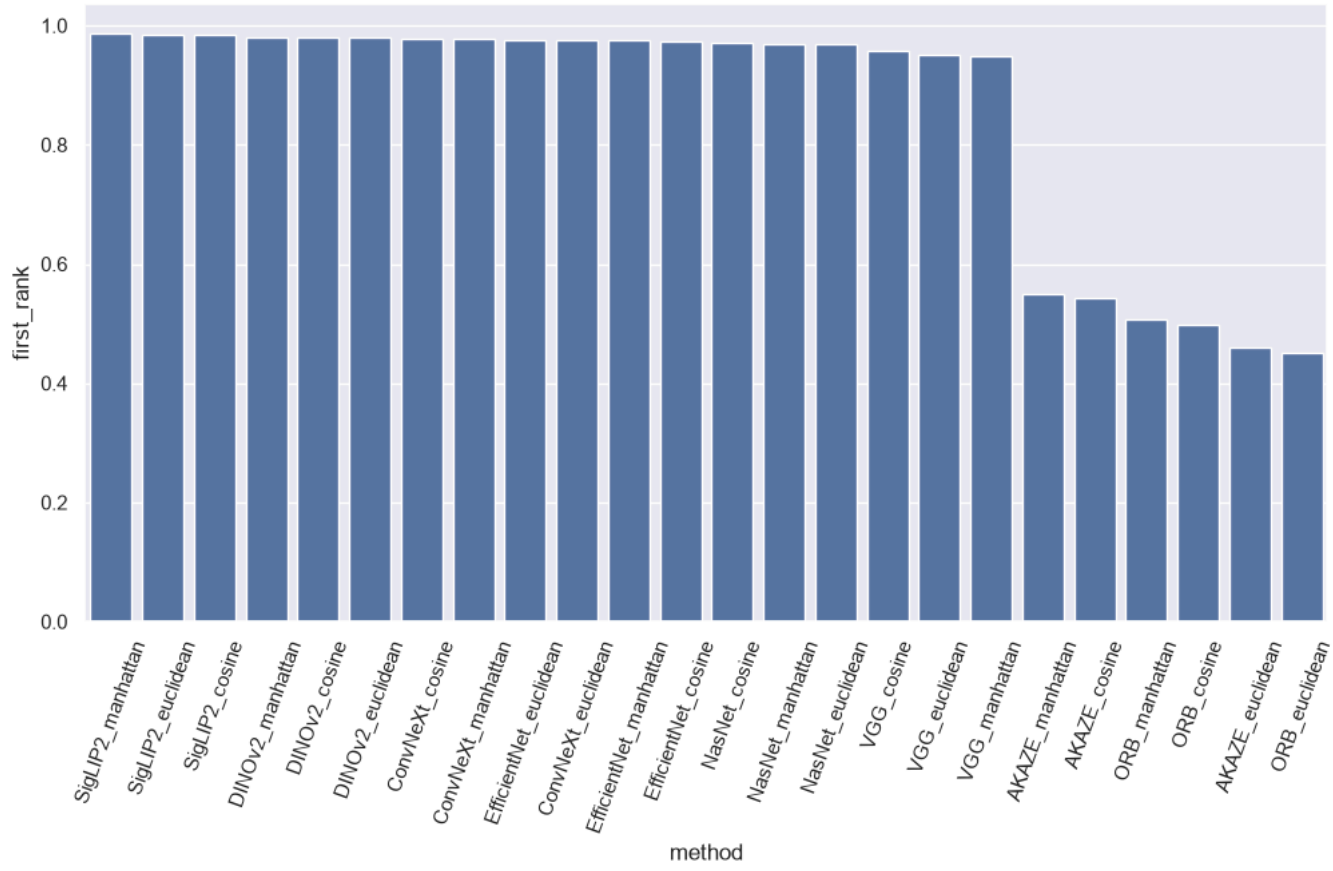

query_ms
0
50
100
150
200
EfficientNet_manhattan
EfficientNet_euclidean
EfficientNet_cosine
NasNet_manhattan
NasNet_euclidean
NasNet_cosine
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
VGG_manhattan
VGG_euclidean
VGG_cosine
SigLIP2_manhattan
SigLIP2_euclidean
SigLIP2_cosine
AKAZE_manhattan
AKAZE_euclidean
AKAZE_cosine
ConvNeXt_manhattan
ConvNeXt_euclidean
ConvNeXt_cosine
ORB_manhattan
ORB_euclidean
ORB_cosine
method

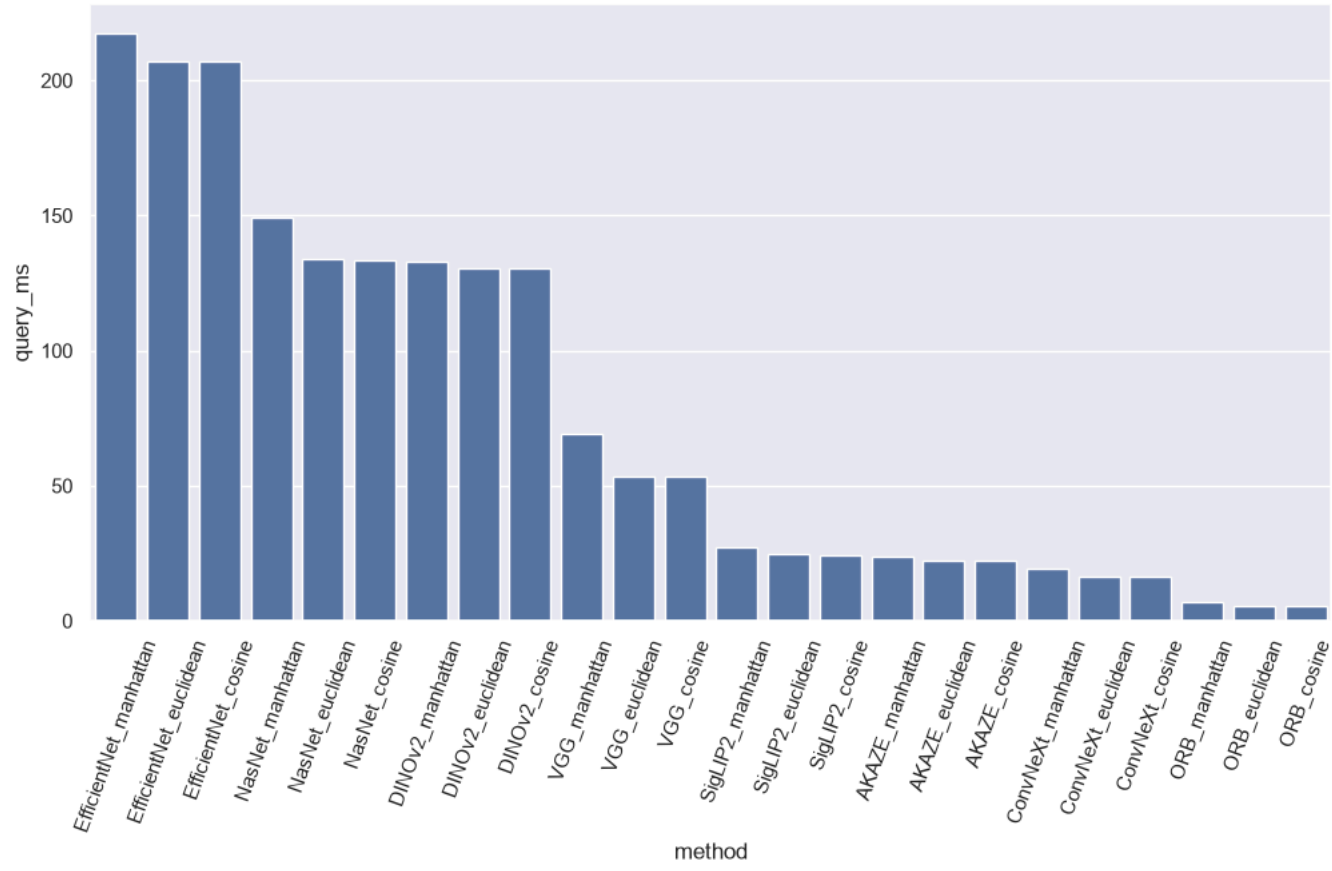
Styles

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.42334	0.650914	0.542044	22.137411
AKAZE_euclidean	0.350433	0.575993	0.460085	22.153636
AKAZE_manhattan	0.435774	0.659575	0.549494	23.839576
ORB_cosine	0.351819	0.626348	0.497605	5.47782
ORB_euclidean	0.306	0.586872	0.450506	5.49867
ORB_manhattan	0.370456	0.633515	0.507185	7.196496
VGG_cosine	0.932021	0.969385	0.957424	53.282498
VGG_euclidean	0.918281	0.965052	0.950239	53.519908
VGG_manhattan	0.917793	0.964955	0.948909	68.997572
NasNet_cosine	0.955753	0.980158	0.971794	133.562377
NasNet_euclidean	0.954304	0.977519	0.968334	133.76933
NasNet_manhattan	0.953868	0.977572	0.969399	149.274009
EfficientNet_cosine	0.96217	0.980912	0.973124	206.975264
EfficientNet_euclidean	0.963613	0.982859	0.976583	207.112004
EfficientNet_manhattan	0.963038	0.982052	0.975785	217.32554
ConvNeXt_cosine	0.96434	0.983808	0.978446	16.464562
ConvNeXt_euclidean	0.964369	0.98238	0.976051	16.498031
ConvNeXt_manhattan	0.964622	0.982961	0.977115	19.152129
DINOv2_cosine	0.969896	0.985737	0.979776	130.376496
DINOv2_euclidean	0.97013	0.985946	0.979776	130.406067
DINOv2_manhattan	0.970741	0.986456	0.981107	133.059558
SigLIP2_cosine	0.969819	0.989693	0.9843	24.507493
SigLIP2_euclidean	0.968002	0.98995	0.984832	24.539723
SigLIP2_manhattan	0.96916	0.991001	0.986429	27.17785


map@5
1.0
0.8
0.6
0.4
0.2
0.0
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
SigLIP2_cosine
SigLIP2_manhattan
SigLIP2_euclidean
ConvNeXt_manhattan
ConvNeXt_euclidean
ConvNeXt_cosine
EfficientNet_euclidean
EfficientNet_manhattan
EfficientNet_cosine
NasNet_cosine
NasNet_euclidean
NasNet_manhattan
VGG_cosine
VGG_euclidean
VGG_manhattan
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method

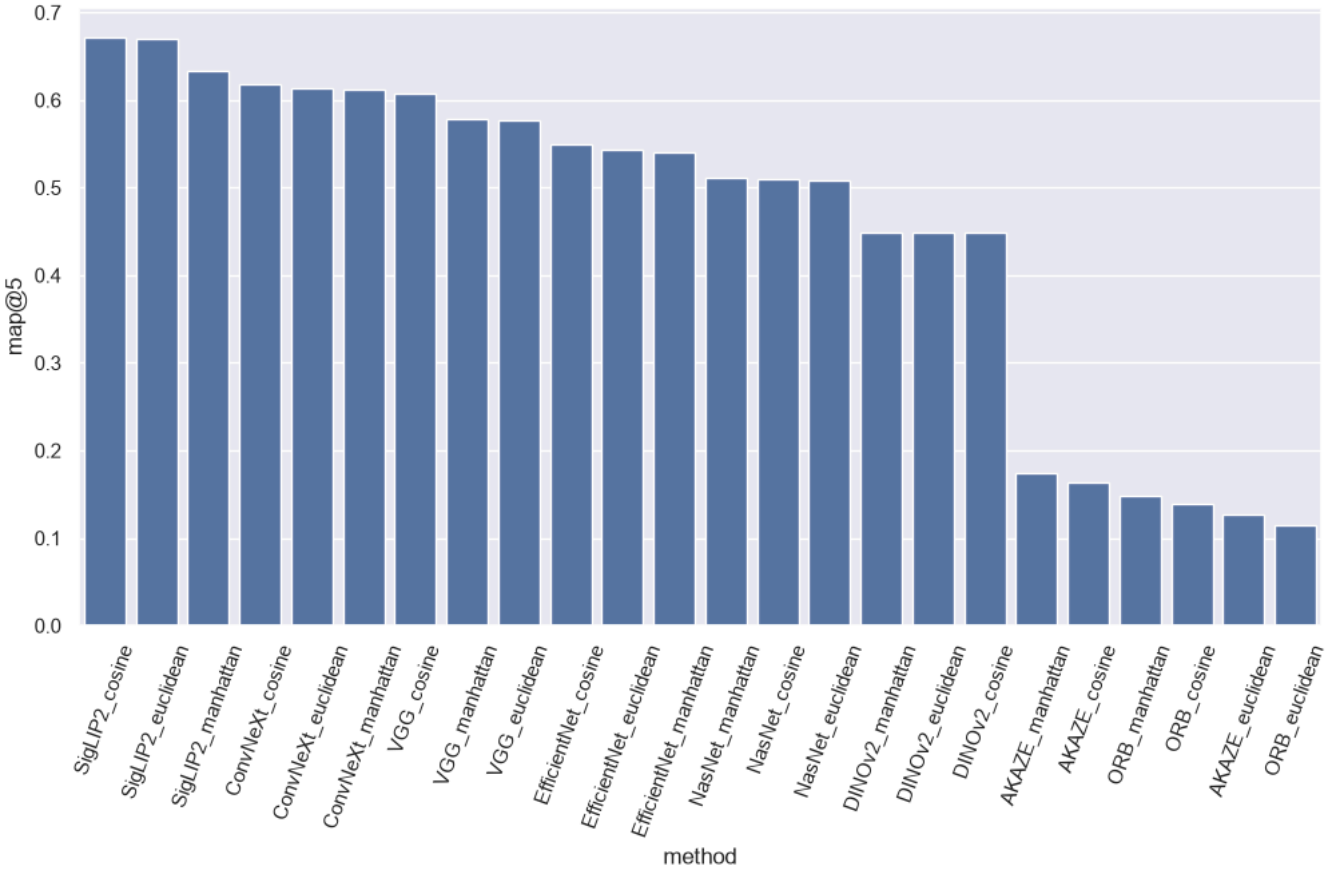

mrr@5
1.0
0.8
0.6
0.4
0.2
0.0
SigLIP2_manhattan
SigLIP2_euclidean
SigLIP2_cosine
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
ConvNeXt_cosine
ConvNeXt_manhattan
EfficientNet_euclidean
ConvNeXt_euclidean
EfficientNet_manhattan
EfficientNet_cosine
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
VGG_cosine
VGG_euclidean
VGG_manhattan
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
ORB_euclidean
AKAZE_euclidean
method

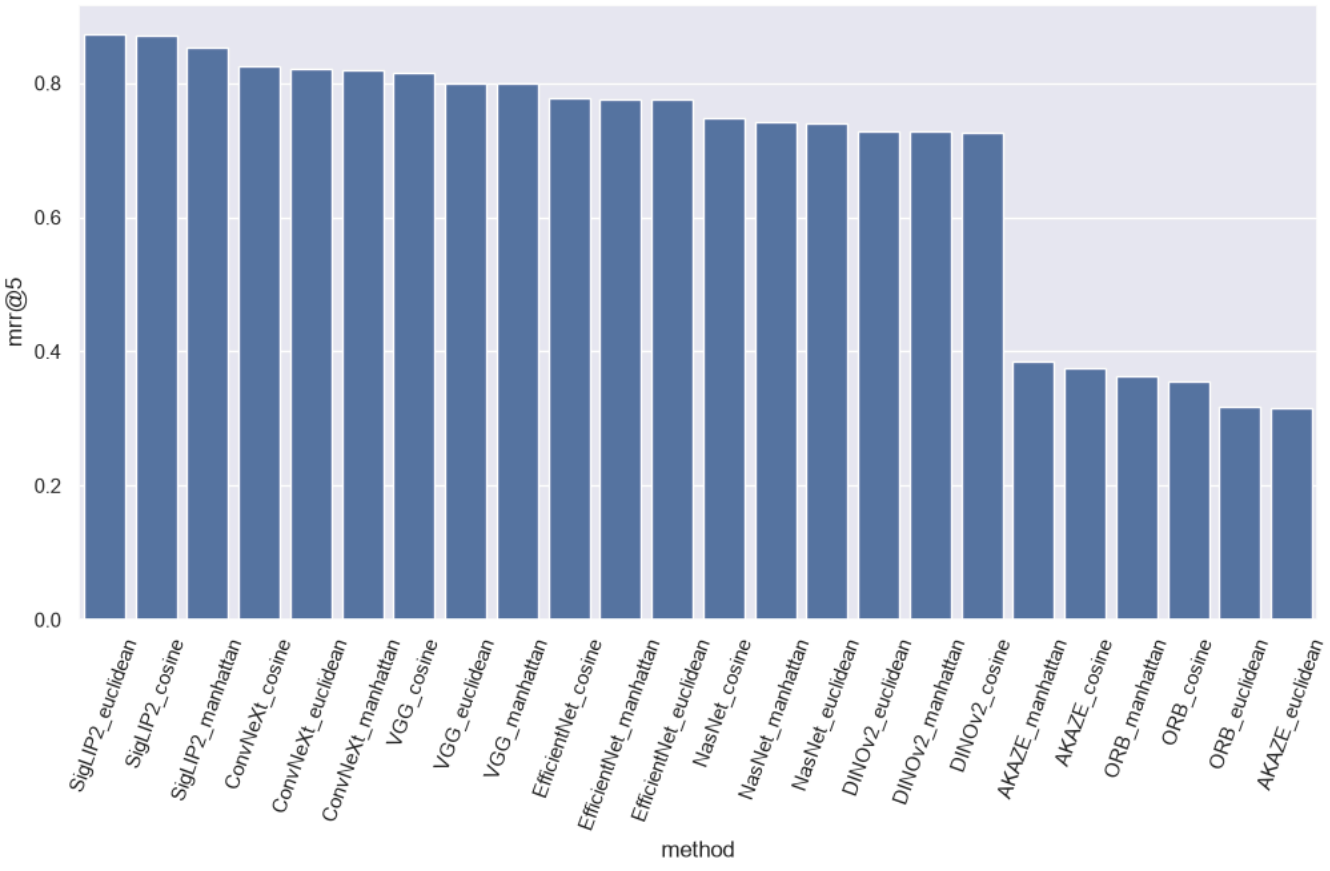


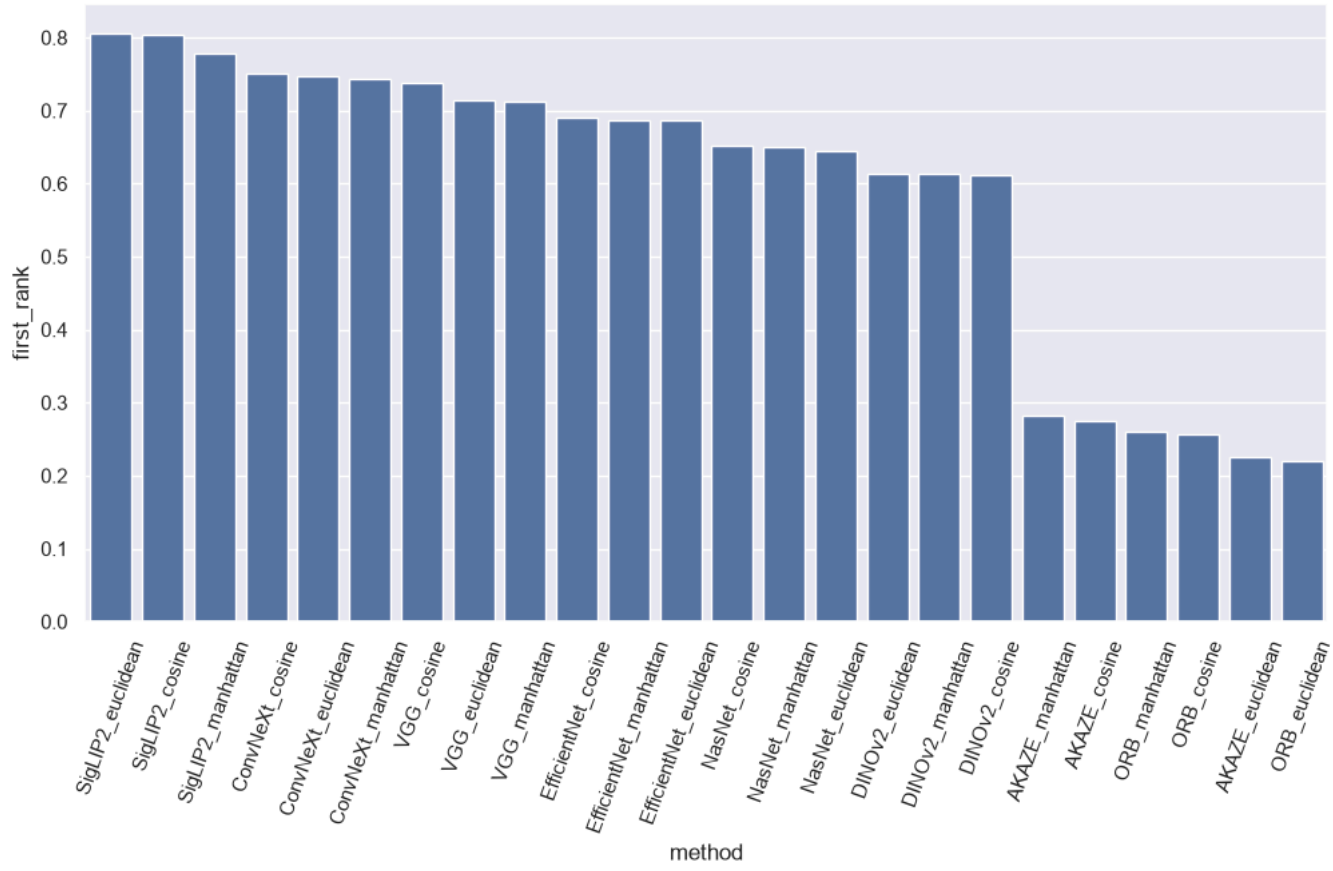


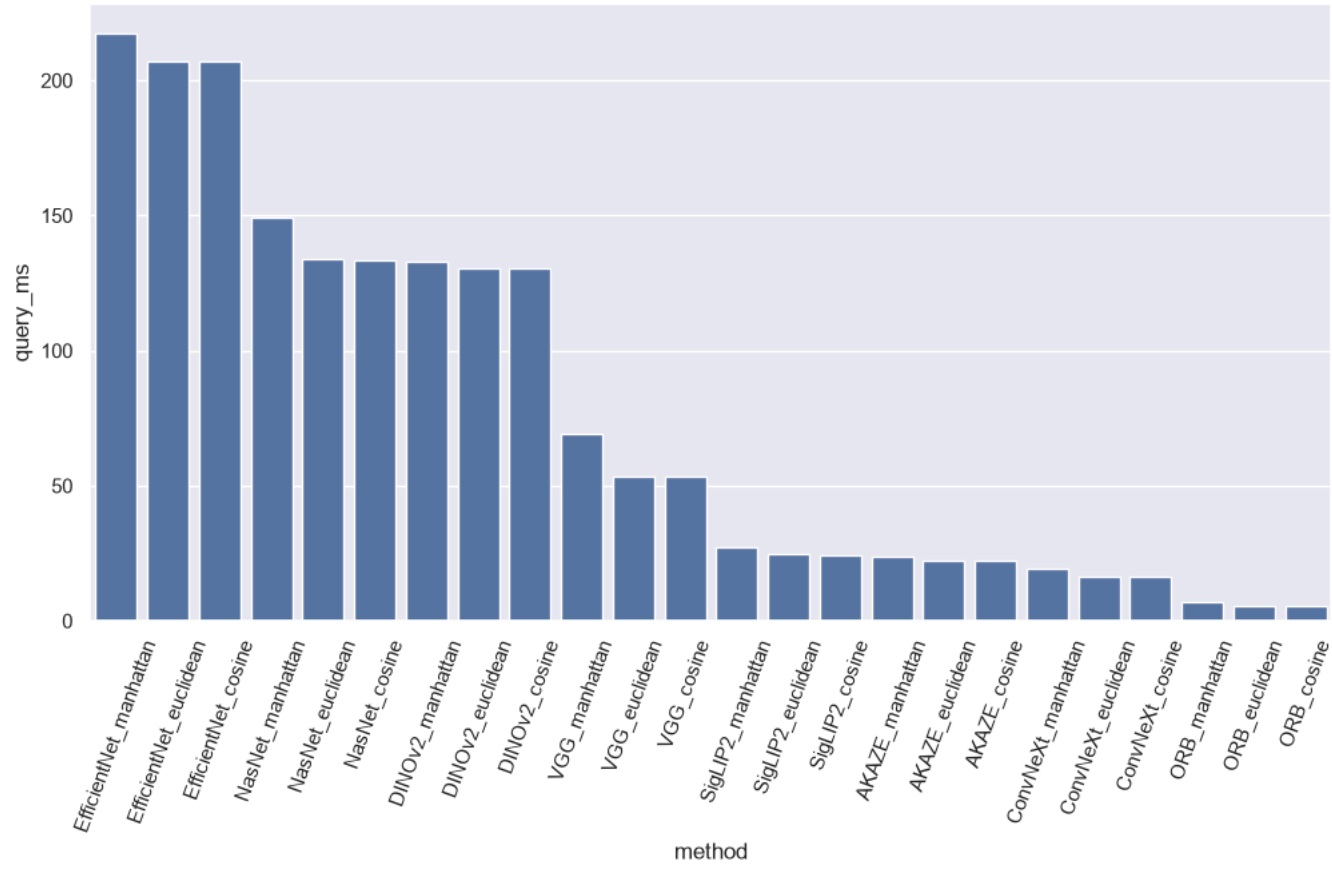
Colors & Styles

method	map@5	mrr@5	first_rank	query_ms
AKAZE_cosine	0.16409	0.375213	0.274082	22.137411
AKAZE_euclidean	0.126223	0.31519	0.225652	22.153636
AKAZE_manhattan	0.174307	0.384575	0.281533	23.839576
ORB_cosine	0.139508	0.35565	0.256519	5.47782
ORB_euclidean	0.114307	0.317279	0.219532	5.49867
ORB_manhattan	0.148775	0.363704	0.259447	7.196496
VGG_cosine	0.607137	0.81574	0.738159	53.282498
VGG_euclidean	0.576673	0.799299	0.715008	53.519908
VGG_manhattan	0.579004	0.799175	0.712879	68.997572
NasNet_cosine	0.510502	0.74737	0.651676	133.562377
NasNet_euclidean	0.50794	0.740828	0.644226	133.76933
NasNet_manhattan	0.511615	0.741365	0.649282	149.274009
EfficientNet_cosine	0.548953	0.778162	0.690261	206.975264
EfficientNet_euclidean	0.543191	0.775302	0.686535	207.112004
EfficientNet_manhattan	0.540621	0.775483	0.6876	217.32554
ConvNeXt_cosine	0.61849	0.826126	0.750665	16.464562
ConvNeXt_euclidean	0.612827	0.821004	0.747206	16.498031
ConvNeXt_manhattan	0.611849	0.818325	0.742682	19.152129
DINOv2_cosine	0.448092	0.726814	0.611495	130.376496
DINOv2_euclidean	0.448297	0.727404	0.613092	130.406067
DINOv2_manhattan	0.448485	0.727266	0.612826	133.059558
SigLIP2_cosine	0.671273	0.869847	0.803087	24.507493
SigLIP2_euclidean	0.670323	0.872485	0.805482	24.539723
SigLIP2_manhattan	0.633123	0.852696	0.778872	27.17785


map@5
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
SigLIP2_cosine
SigLIP2_euclidean
SigLIP2_manhattan
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_cosine
VGG_manhattan
VGG_euclidean
EfficientNet_cosine
EfficientNet_euclidean
EfficientNet_manhattan
NasNet_manhattan
NasNet_cosine
NasNet_euclidean
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method


mrr@5
0.0
0.2
0.4
0.6
0.8
SigLIP2_euclidean
SigLIP2_cosine
SigLIP2_manhattan
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_cosine
VGG_euclidean
VGG_manhattan
EfficientNet_cosine
EfficientNet_manhattan
EfficientNet_euclidean
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
DINOv2_euclidean
DINOv2_manhattan
DINOv2_cosine
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
ORB_euclidean
AKAZE_euclidean
method


first_rank
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
SigLIP2_euclidean
SigLIP2_cosine
SigLIP2_manhattan
ConvNeXt_cosine
ConvNeXt_euclidean
ConvNeXt_manhattan
VGG_cosine
VGG_euclidean
VGG_manhattan
EfficientNet_cosine
EfficientNet_manhattan
EfficientNet_euclidean
NasNet_cosine
NasNet_manhattan
NasNet_euclidean
DINOv2_euclidean
DINOv2_manhattan
DINOv2_cosine
AKAZE_manhattan
AKAZE_cosine
ORB_manhattan
ORB_cosine
AKAZE_euclidean
ORB_euclidean
method


query_ms
0
50
100
150
200
EfficientNet_manhattan
EfficientNet_euclidean
EfficientNet_cosine
NasNet_manhattan
NasNet_euclidean
NasNet_cosine
DINOv2_manhattan
DINOv2_euclidean
DINOv2_cosine
VGG_manhattan
VGG_euclidean
VGG_cosine
SigLIP2_manhattan
SigLIP2_euclidean
SigLIP2_cosine
AKAZE_manhattan
AKAZE_euclidean
AKAZE_cosine
ConvNeXt_manhattan
ConvNeXt_euclidean
ConvNeXt_cosine
ORB_manhattan
ORB_euclidean
ORB_cosine
method

Conclusion

Les réseaux de neurones surpassent nettement les descripteurs : sur la couleur et le style ensemble, leur MAP@5 va de 0,45 à 0,67, contre 0,11 à 0,17 pour AKAZE et ORB.

SigLIP 2, pré-entraîné sur des paires image-texte, est le meilleur sur la couleur et le style (MAP@5 0,671 en distance cosinus) et sur la couleur seule (0,689), pour environ 25 ms par requête. ConvNeXt-Tiny vient ensuite (0,618) et c'est le réseau le plus rapide (environ 16 ms). Les gros réseaux de l'étude d'origine, EfficientNet-B7 et NASNetLarge, sont à la fois moins précis et 5 à 13 fois plus lents que ces deux-là.

DINOv2 a le meilleur MAP@5 sur le style seul (0,971) mais le moins bon des réseaux sur la couleur (0,458) : ses vecteurs décrivent la forme plutôt que la couleur. Une explication possible, non vérifiée ici : son entraînement auto-supervisé utilise des variations de couleur aléatoires, ce qui lui apprend à ignorer la couleur.

La mesure compte peu pour les réseaux : sur la couleur et le style, la distance cosinus est la meilleure ou à moins de 0,005 de la meilleure pour chaque réseau ; seule la distance de Manhattan avec SigLIP 2 décroche (0,633 contre 0,671). Pour AKAZE et ORB, cosinus et Manhattan font jeu égal, devant l'euclidienne.

Le temps d'une requête vient presque entièrement de l'extraction. La recherche exacte sur 7 627 images coûte moins de 1 ms en distance cosinus ou euclidienne, et 3 à 16 ms en distance de Manhattan selon la taille des vecteurs.